FACEBLUR — Training Scripts Guide

How to use `train_all.py` and `train_head.py` to train head-detection models

What each script is for

Script	Trains	Use it when
`train_all.py`	Several sizes (nano/small/medium) back-to-back	You want to **compare sizes** or ship a size selector. One command, unattended, produces `head_n.pt` / `head_s.pt` / `head_m.pt` + a comparison.
`train_head.py`	**One** model, with an optional hyperparameter search first	You want the **best single model**, and you're willing to spend time on an automated tune before a long final run.

Both are for the same goal — a `head.pt` for FACEBLUR — just different strategies. For most FACEBLUR work, start with `train_all.py`: it's simpler, and it tells you which size actually performs best on your data (which, per our testing, is often **not** the biggest one).

Before you run either (prerequisites)

1. **Activate the training env:** `conda activate yolotrain`
2. **Confirm the GPU is really used** (a mismatch silently falls back to CPU and is ~100x slower): `python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"` You want `True` and your GPU name (e.g. `NVIDIA GeForce RTX 3060`).
3. **Confirm the dataset split is clean** (clip-level, no leakage): `python check_split.py --root head_dataset`
4. **For unattended runs, disable Windows sleep:** `powercfg /change standby-timeout-ac 0`

`train_all.py` — train nano/small/medium in one go

```
python train_all.py --data head_dataset/data.yaml
```

That trains `yolo11n`, `yolo11s`, `yolo11m` in sequence at imgsz 960, copies each to `head_n.pt` / `head_s.pt` / `head_m.pt`, evaluates each, and writes `training_summary.txt` comparing them.

Flags

Flag	Default	Meaning
`--data`	*(required)*	Path to `data.yaml`
`--models`	`n s m`	Which sizes to train (from `n s m l x`)
`--imgsz`	`960`	Training size — **must match the app's** `HEAD_INFER_IMGSZ`
`--epochs`	`300`	Epoch ceiling per model
`--patience`	`60`	Early-stop after this many epochs with no val gain
`--batch`	`-1`	`-1` = auto-size to VRAM per model (recommended)
`--device`	`0`	CUDA index, or `cpu`
`--out`	`.`	Where to copy the renamed `head_<size>.pt`

Examples

```
REM just nano and small (skip the heavy medium)
python train_all.py --data head_dataset/data.yaml --models n s

REM shorter runs that stop nearer the peak (see "Tips" — models overfit late)
python train_all.py --data head_dataset/data.yaml --models n s --epochs 80 --patience 25
```

What you get

- `head_n.pt`, `head_s.pt`, `head_m.pt` in `--out`
- `training_summary.txt` — a recall/mAP comparison table
- Per-model runs under `runs/detect/head_<size>/` (curves in `results.png`, weights in `weights/best.pt`)

Robust for unattended runs: if one size errors out, it logs the failure and continues to the next, so you still get the others.

`train_head.py` — one model, with an automated tune

```
python train_head.py --data head_dataset/data.yaml --final-imgsz 960
```

Runs in two phases: 1. **Search** — ~15 short trials at low resolution to find good augmentation / learning-rate values (fast). 2. **Final** — one long run at full resolution using those values, then a test-split evaluation.

Flags

Flag	Default	Meaning
`--data`	*(required)*	Path to `data.yaml`
`--model`	`yolo11s.pt`	Base weights to start from
`--name`	`head_final`	Run name
`--no-tune`	off	Skip the search phase (go straight to the final run)
`--tune-imgsz`	`640`	Resolution during the search (small = fast)
`--tune-batch`	`16`	Batch during the search
`--tune-epochs`	`20`	Epochs per search trial
`--iterations`	`15`	Number of search trials
`--final-imgsz`	`1280`	Final-run resolution — **see the warning below**
`--final-batch`	`8`	Final-run batch
`--epochs`	`300`	Final-run epoch ceiling
`--patience`	`60`	Early-stop patience
`--device`	`0`	CUDA index, or `cpu`
`--no-test`	off	Skip the held-out test evaluation

> ⚠ **Override `--final-imgsz` to 960 for FACEBLUR**
>
> `train_head.py` defaults `--final-imgsz` to **1280**, but the FACEBLUR app runs the head model at `HEAD_INFER_IMGSZ = 960`. **Train and run at the same size**, or you get scale-mismatch false positives (the weapon-illuminator problem). So for FACEBLUR, pass `--final-imgsz 960` — unless you also change `HEAD_INFER_IMGSZ` to 1280 in `face_blur.py` and accept slower CPU inference.

Examples

```
REM full plan, matched to the app
python train_head.py --data head_dataset/data.yaml --final-imgsz 960

REM skip the search, just one long run
python train_head.py --data head_dataset/data.yaml --final-imgsz 960 --no-tune

REM train a nano model instead of the default small
python train_head.py --data head_dataset/data.yaml --model yolo11n.pt --final-imgsz 960
```

What you get

- `runs/detect/head_final/weights/best.pt` — rename to `head.pt` to ship
- Printed recall / mAP on the test split

Which should I use?

- **Deciding which size to ship, or shipping the size selector →** `train_all.py`.
- **You already know the size and want the single strongest model →** `train_head.py` (start from `yolo11n.pt` or `yolo11s.pt`, `--final-imgsz 960`).
- **Quick iteration between labeling rounds →** `train_all.py --models n s --epochs 80` (fast, and nano/small are what perform best here anyway).

After training — deploy the model

1. Pick the winner from `training_summary.txt` (or `head_final/best.pt`).
2. **Confirm it's `best.pt`, not `last.pt`** — `best.pt` is saved at the peak epoch; `last.pt` is the (often overfit) final epoch.
3. Rename to `head_n.pt` / `head_s.pt` / `head_m.pt` (or `head.pt`) and place in the project root, then rebuild. The installer bundles them next to the exe.
4. Set `HEAD_INFER_IMGSZ` in `face_blur.py` to the size you trained at (960).

Tips (from testing on this dataset)

- **Bigger is not better here.** On our data, nano beat small beat medium on the hard (catwalk) domain — the larger models **overfit** the limited dataset. Check each run's `results.png`: if `val/box_loss` turns upward while `train` keeps dropping, that's overfitting. Prefer the smaller model unless you've added a lot more data.
- **Models peak early, then drift.** Val metrics often peak well before the epoch ceiling. `best.pt` captures the peak, but you can also stop sooner (`--patience 25`, `--epochs 60-80`) to save time.
- **The bottleneck is data, not model size or epochs.** If recall is low on a domain, add more (hard) labeled frames for that domain — you can't out-train a data limitation.
- **Always confirm the GPU banner** at the start of a run: `CUDA:0 (Your GPU …)`, not `CPU`. Watch the `GPU_mem` column stays under your VRAM (else it's spilling to system RAM and running slow).
- **Evaluate per domain** afterward with `eval_domains.py` — a pooled score hides one domain underperforming behind another.

FACEBLUR training scripts — `train_all.py`, `train_head.py`.